SolarVision-MLOps

Segmentation de toitures par Vision par Ordinateur pour estimer le potentiel solaire des batiments

Projet de Fin de Module - Industrialisation Vision & MLOps de bout en bout

Rapport	Aide a la decision du sujet de projet
Axe retenu	Axe 3 : Detection d'objets & segmentation en temps reel (YOLO11-seg), avec comparaison SegFormer (axe 4)
Equipe	Trinome (3 etudiants) - roles definis en section 8
Stack	FastAPI - React - DVC - MLflow - ONNX - Docker Compose - GitHub Actions

Avertissement. Ce rapport a ete prepare sans acces a Internet ni a une base bibliographique. Les references de la section 12 correspondent a des travaux reels et connus, mais les details (pages, DOI, annees exactes) doivent etre verifies avant tout depot. Le calendrier et le bareme officiels sont ceux du cahier des charges PDF du module.

1. Resume executif

Le document de depart montre qu'un batiment peut produire sa propre energie (panneaux photovoltaiques, stockage) et que la Data Science peut aider les architectes a concevoir des batiments plus sobres. Mais le module impose un projet de **Vision par Ordinateur industrialise** : il faut donc transformer cette idee 'energie' en un probleme d'**images**.

Notre proposition, **SolarVision-MLOps**, repond a cette contrainte : a partir d'une image aerienne ou satellite d'un batiment, le systeme **segmente automatiquement la toiture et ses obstacles** (cheminees, climatiseurs, ombres), calcule la **surface utile** en m2, puis estime la **production solaire annuelle** et la compare a la consommation du batiment. L'architecte obtient en quelques secondes un premier avis de faisabilite 'batiment a energie positive'.

Pourquoi ce choix ? Il satisfait 100 % du cahier des charges (axe thematique, stack obligatoire, soutenance pratique), il s'appuie sur des jeux de donnees publics, il se demontre facilement en direct (upload, webcam, masques dynamiques) et il possede une vraie valeur applicative.

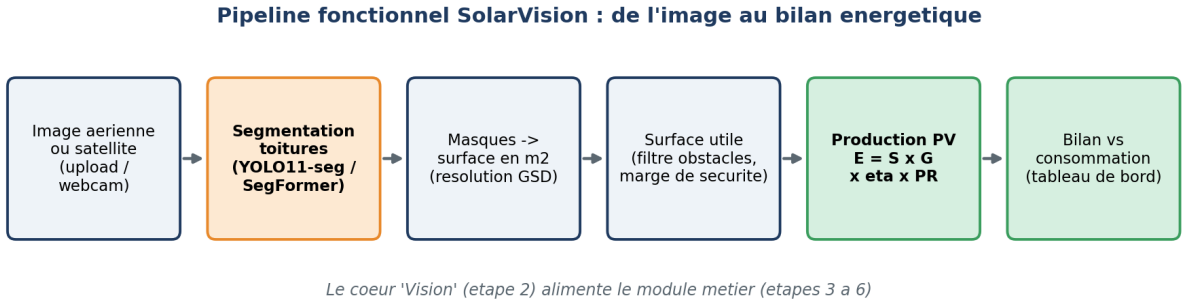


Figure 1 - Pipeline fonctionnel : de l'image au bilan energetique

2. Contexte et problematique

Un batiment a energie positive produit plus d'energie qu'il n'en consomme sur une annee. Les sources principales sont le photovoltaique, le solaire thermique, les petites eoliennes et le stockage par batteries. Pour un projet urbain, la premiere question de l'architecte est simple : *'Quelle surface de toiture puis-je reellement equiper ?'*

Aujourd'hui, cette surface est mesuree a la main sur des plans ou des images, ce qui est lent, couteux et peu reproductible a grande echelle. La litterature montre que la segmentation par apprentissage profond permet de cartographier automatiquement les toitures et les installations photovoltaiques (Malof et al., 2016 ; Yu et al., 2018 ; Kruitwagen et al., 2021).

Problematique : comment concevoir, entrainer, industrialiser et deployer un systeme de vision capable de segmenter des toitures en quasi temps reel et d'en deduire un potentiel solaire exploitable par un architecte ?

Lien avec le document d'origine. Le document propose un assistant 'Smart Building AI' qui predit la consommation a partir de tableaux de caracteristiques (surface, orientation, isolation). SolarVision conserve cet objectif mais remplace l'entree tabulaire par une **entree image**, ce qui rend le projet compatible avec le module Vision. Le module de consommation (XGBoost) peut rester un enrichissement facultatif.

3. Conformite avec le cahier des charges

Exigence du module	Reponse de SolarVision-MLOps
Format trinome (1 a 3 etudiants)	Equipe de 3 ; repartition des roles en section 8
Axe thematique au choix	Axe 3 (detection & segmentation temps reel, YOLO11-seg) + comparaison SegFormer (axe 4) comme etude complementaire
Backend FastAPI asynchrone, Pydantic, Swagger /docs	Endpoints /predict (image), /predict/stream (webcam), /energy, /health ; schemas Pydantic ; /docs automatique
Frontend React ou Angular : upload, webcam, affichage dynamique	Application React : upload, capture webcam, superposition des masques et bounding boxes, tableau de bord kWh
DVC (versioning des donnees)	Dataset d'images et d'annotations versionne ; pipeline dvc.yaml (prepare, train, evaluate)
MLflow (tracking + Model Registry)	Suivi des runs (mAP, mIoU, latence) ; promotion Staging vers Production
ONNX (optimisation / deploiement)	Export du meilleur modele ; comparaison de latence PyTorch vs ONNX Runtime
Docker Compose	Services : frontend, api, mlflow, postgres
GitHub Actions (CI/CD)	Lint, tests, build des images, verification de l'export ONNX
Soutenance pratique et technique	Demo en direct + execution de la chaine, section 10

4. Aide a la decision : choix de l'axe thematique

Nous avons evalue les quatre axes sur cinq criteres (note de 1 a 5, poids egaux) : pertinence avec le theme energie/batiment, disponibilite des donnees, faisabilite (temps et GPU), potentiel de demonstration temps reel, et couverture de la stack MLOps imposee.

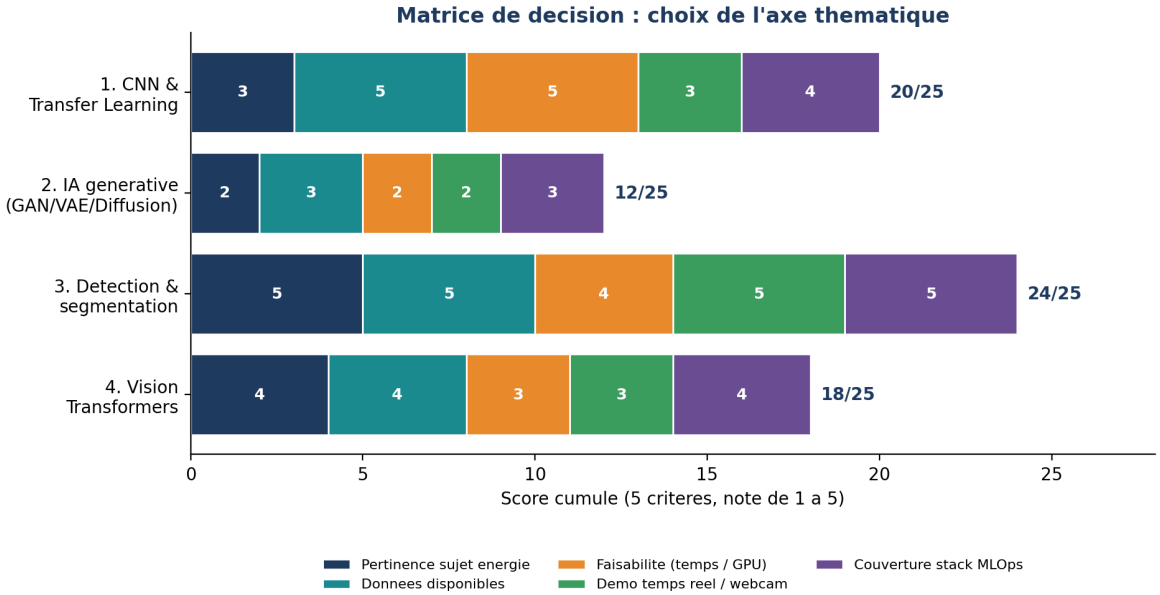


Figure 2 - Matrice de decision (scores a discuter en equipe)

Axe	Avantages	Limites pour ce sujet
1. CNN & Transfer Learning	Simple, rapide a entrainer, tres documente	Classification seule : peu parlante pour mesurer une surface
2. IA generative	Spectaculaire	Evaluation difficile, GPU lourd, lien energie faible

Axe	Avantages	Limites pour ce sujet
3. Detection & segmentation	**Masques = surfaces mesurables ; temps reel ; demo webcam naturelle**	Annotation de masques a gerer (mitigee par datasets publics)
4. Vision Transformers	Etat de l'art (SegFormer)	Plus gourmand ; retenu comme comparaison secondaire

Decision proposee : axe 3 comme axe principal, avec SegFormer en benchmark. Cette combinaison montre la maitrise de deux familles de modeles (CNN temps reel vs Transformer) sans multiplier les risques.

5. Description du projet SolarVision-MLOps

5.1 Objectifs

- Segmenter les toitures (classe principale) et les obstacles sur images aeriennes/satellite.
- Convertir les masques en surface reelle (m2) grace a la resolution au sol de l'image (GSD, en m/pixel).
- Estimer la production photovoltaique annuelle et la comparer a la consommation du batiment.
- Industrialiser l'ensemble : donnees versionnees, experiences tracees, modele optimise, deploiement reproductible, CI/CD.

5.2 Donnees

Plusieurs jeux publics sont envisageables : **Inria Aerial Image Labeling** (masques de batiments sur plusieurs villes), **jeu de donnees de panneaux PV de Bradbury et al. (2016)**, **WHU Building Dataset** (Ji et al., 2019), ainsi que SpaceNet. Un petit jeu complementaire annote par l'equipe (batiments de Tunisie, via captures cartographiques respectant les licences) permettrait d'evaluer la generalisation locale. *Les licences et conditions d'usage de chaque jeu doivent etre verifiees.*

5.3 Modeles

Modele	Role	Pourquoi
YOLO11-seg (Ultralytics)	Modele principal, temps reel	Detection + segmentation d'instances en une passe ; export ONNX natif ; entrainement rapide
SegFormer (axe 4)	Benchmark Transformer	Segmentation semantique efficace ; comparaison precision/latence
U-Net (reference)	Baseline classique	Reference historique en segmentation d'images aeriennes

5.4 Estimation energetique

La production annuelle est estimee par la formule classique : **E = S x G x eta x PR**, ou S est la surface utile (m2), G l'irradiation annuelle (kWh/m2/an), eta le rendement des modules et PR le ratio de performance (pertes systeme). Exemple illustratif pour une toiture a Tunis, avec G = 1800 kWh/m2/an (ordre de grandeur a confirmer avec l'outil PVGIS), eta = 20 % et PR = 0,8 : 100 m2 utiles produisent environ 100 x 1800 x 0,20 x 0,80 = **28 800 kWh/an**. Pour couvrir 45 000 kWh/an, il faudrait environ 156 m2 utiles.

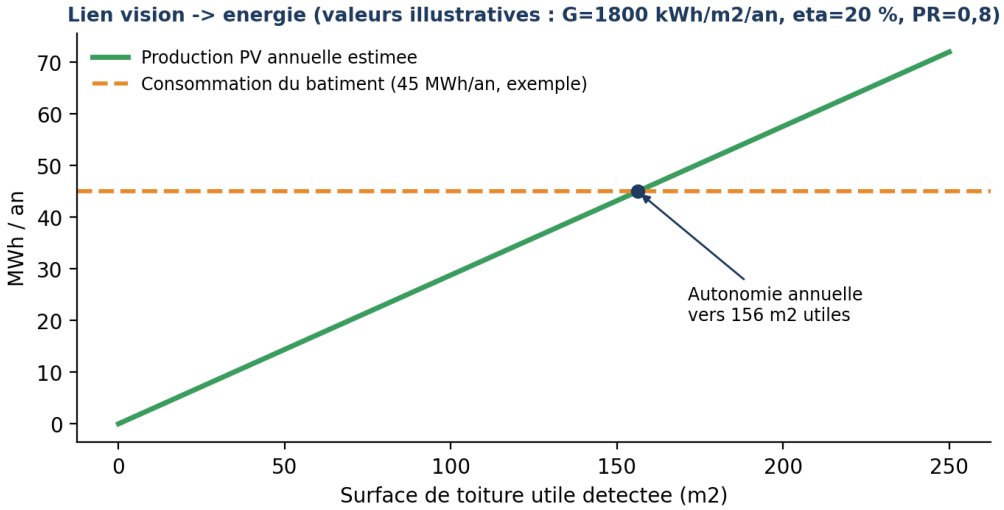


Figure 3 - Lien entre la surface detectee par la vision et la production annuelle (valeurs illustratives)

6. Architecture technique

L'application est decoupee en services conteneurises. Le frontend React communique avec l'API FastAPI, qui appelle le module d'inference ONNX Runtime puis le module energie. PostgreSQL conserve l'historique des analyses. MLflow gere le suivi des experiences et fournit le modele marque 'Production'.

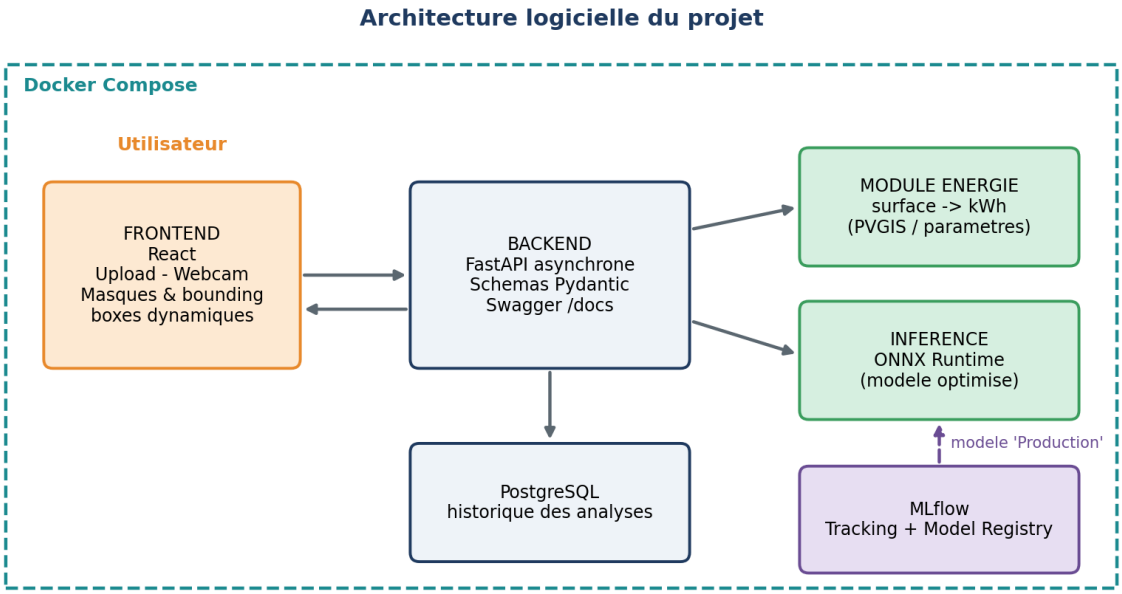


Figure 4 - Architecture logicielle orchestree par Docker Compose

Choix de conception importants

- **API asynchrone** : les endpoints d'inference utilisent async/await ; le calcul lourd est delegue a l'executeur pour ne pas bloquer la boucle d'evenements.
- **Schemas Pydantic** : validation stricte des entrees (formats d'image, GSD, parametres energetiques) et des sorties (masques, classes, scores, kWh).
- **Flux webcam** : le frontend capture des images a cadence reduite et les envoie a l'API (ou via WebSocket) ; les masques sont redessines sur un canvas.

- **Separation donnees / modele / code** : DVC pour les donnees, MLflow pour les modeles, Git pour le code.

7. Chaine MLOps de bout en bout

La litterature insiste sur le fait que le modele ne represente qu'une petite partie d'un systeme ML en production, le reste etant de l'infrastructure, du suivi et de la gestion des donnees (Sculley et al., 2015 ; Kreuzberger et al., 2023). La chaine ci-dessous reflete cette vision.

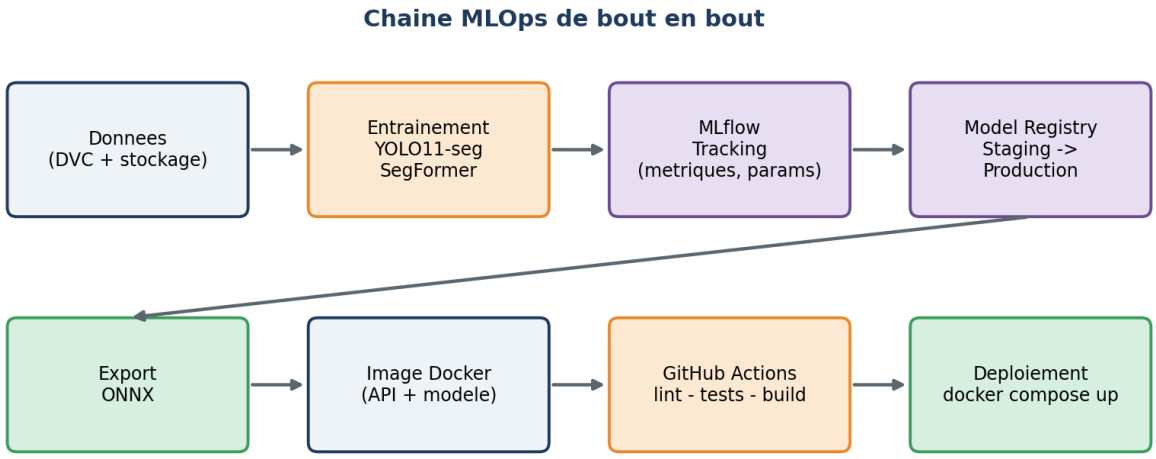


Figure 5 - Chaine MLOps : donnees, entrainement, registre, ONNX, Docker, CI/CD

Outil	Usage dans le projet	Livrable verifiable en soutenance
DVC	Versionner images, masques, splits ; pipeline reproductible	dvc repro ; historique des versions de donnees
MLflow	Tracer hyperparametres, mAP, mIoU, latence ; registre de modeles	Interface MLflow avec comparaison de runs
ONNX	Exporter et accelerer l'inference	Tableau de latence PyTorch vs ONNX Runtime
Docker Compose	Orchestrer frontend, API, MLflow, base de donnees	docker compose up : application fonctionnelle
GitHub Actions	Lint, tests unitaires, build d'images, controle de l'export	Pipeline vert sur le depot

8. Organisation de l'equipe et planning

Role	Responsabilites principales
Etudiant A - Data & Modeles	Datasets, DVC, entrainement YOLO11-seg et SegFormer, suivi MLflow, evaluation
Etudiant B - Backend & MLOps	FastAPI, Pydantic, export ONNX, Docker Compose, GitHub Actions
Etudiant C - Frontend & Energie	Application React (upload, webcam, masques), module energie, tableau de bord

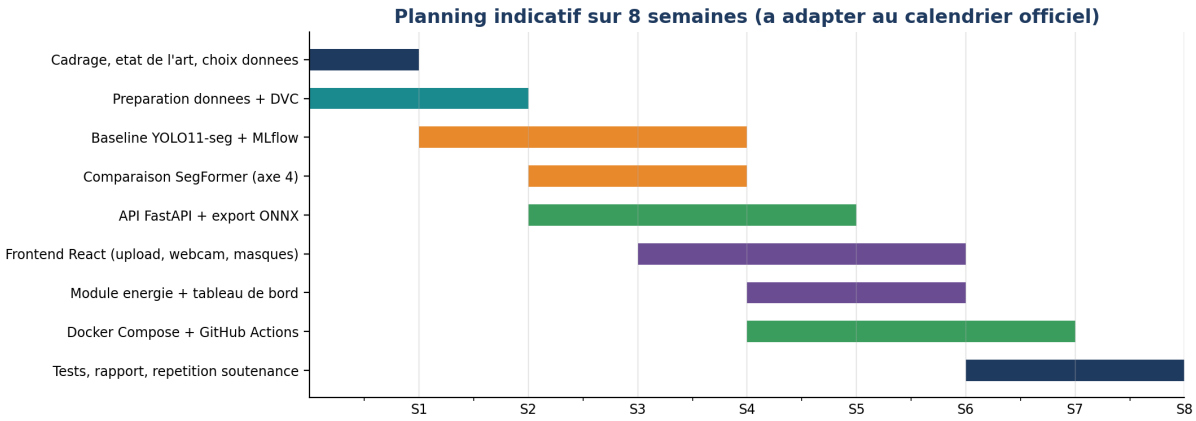


Figure 6 - Planning indicatif (a recaler sur le calendrier officiel du module)

Chaque membre doit maitriser l'ensemble de la chaine : la soutenance etant technique, des questions croisees sont probables.

9. Metriques d'evaluation

Dimension	Metrique	Objectif indicatif
Qualite de segmentation	mIoU, Dice, mAP masques	Superieur a la baseline U-Net
Qualite metier	Erreur relative sur la surface en m2	Erreur moyenne inferieure a 10 % sur le jeu de test
Performance	Latence par image, images/s (PyTorch vs ONNX)	Gain mesure et documente grace a ONNX
Robustesse	Resultats par ville / saison / resolution	Analyse des cas d'echec (ombres, toits sombres)
Industrialisation	CI verte, reproductibilite (dvc repro)	Reconstruction complete a partir d'un clone propre

Les objectifs chiffres sont indicatifs et doivent etre ajustes apres la baseline.

10. Plan de soutenance pratique

- **1. Demarrage (2 min)** : clone propre, dvc pull, docker compose up ; ouverture de /docs (Swagger).
- **2. Demo vision (5 min)** : upload d'une image, puis flux webcam (image aerienne affichee sur ecran ou video de drone) ; masques et bounding boxes dynamiques.
- **3. Demo energie (2 min)** : surface utile, kWh estimes, bilan par rapport a la consommation.
- **4. MLOps (5 min)** : comparaison de runs MLflow, promotion d'une version dans le Model Registry, tableau PyTorch vs ONNX.
- **5. CI/CD (2 min)** : pipeline GitHub Actions, modification mineure du code et nouvelle execution.
- **6. Questions techniques** : preparer des reponses sur les choix de modele, les limites et les risques.

11. Risques et argumentaire pour convaincre l'equipe

Risque	Mitigation
Annotation de masques trop longue	Partir de datasets publics ; pre-annotation avec un modele de segmentation general (Segment Anything) ; petit jeu local seulement pour l'evaluation
Manque de GPU	Modeles legers (variantes small/nano), Google Colab / Kaggle, images redimensionnees

Risque	Mitigation
Generalisation faible a la Tunisie	Fine-tuning sur jeu local, analyse d'erreurs documentee, augmentation de donnees
Integration tardive de la stack	CI et Docker des la semaine 3 ; livrer d'abord un squelette bout en bout
Estimation energetique trop simpliste	Presenter clairement les hypotheses ; possibilite d'ajouter PVGIS en option

Arguments cles pour les collaborateurs

- **Conformite totale** : chaque exigence du cahier des charges a une reponse concrete et verifiable.
- **Faisabilite** : donnees publiques, modeles pre-entraines, outils matures ; pas de dependance a un materiel exceptionnel.
- **Demo convaincante** : un jury voit tout de suite l'image, les masques, les kWh et la chaine MLOps.
- **Valeur reelle** : cas d'usage concret pour architectes, collectivites et bureaux d'etudes en transition energetique.
- **Evolutivite** : ajout possible du module consommation (XGBoost), de PVGIS ou de la detection de panneaux deja installes.

12. References scientifiques

Important : ces references sont donnees de memoire, sans acces a une base bibliographique. Il faut verifier chaque entree (titre, auteurs, annee, pages, DOI) sur Google Scholar, arXiv ou le site de l'editeur, puis telecharger les PDF correspondants.

Segmentation, detection et architectures

[1] Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.

[2] He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR.

[3] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.

[4] Ren, S., He, K., Girshick, R., Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS.

[5] He, K., Gkioxari, G., Dollar, P., Girshick, R. (2017). Mask R-CNN. ICCV.

[6] Terven, J., Cordova-Esparza, D. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction.

[7] Kirillov, A. et al. (2023). Segment Anything. ICCV.

Vision Transformers et transfert

[8] Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR.

[9] Liu, Z. et al. (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. ICCV.

[10] Xie, E. et al. (2021). SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. NeurIPS.

[11] Pan, S. J., Yang, Q. (2010). A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering.

Imagerie aerienne et potentiel solaire

[12] Malof, J. M., Bradbury, K., Collins, L. M., Newell, R. G. (2016). Automatic detection of solar photovoltaic arrays in high resolution aerial imagery. Applied Energy.

[13] Yu, J., Wang, Z., Majumdar, A., Rajagopal, R. (2018). DeepSolar: A Machine Learning Framework to Efficiently Construct a Solar Deployment Database in the United States. Joule.

[14] Kruitwagen, L. et al. (2021). A global inventory of photovoltaic solar energy generating units. Nature.

[15] Bradbury, K. et al. (2016). Distributed solar photovoltaic array location and extent data set for remote sensing object identification. Scientific Data.

[16] Maggiori, E., Tarabalka, Y., Charpiat, G., Alliez, P. (2017). Can semantic labeling methods generalize to any city? The Inria aerial image labeling benchmark. IGARSS.

[17] Ji, S., Wei, S., Lu, M. (2019). Fully Convolutional Networks for Multisource Building Extraction from an Open Aerial and Satellite Imagery Data Set. IEEE Transactions on Geoscience and Remote Sensing.

MLOps

[18] Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS.

[19] Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. IEEE Data Engineering Bulletin.

[20] Kreuzberger, D., Kuhl, N., Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access.

Total : 20 references (minimum demande : 10). Les outils logiciels (Ultralytics YOLO11, FastAPI, DVC, MLflow, ONNX Runtime, PVGIS) sont a citer via leur documentation officielle.

Annexe A - Bibliotheque d'articles telechargeables

Cette annexe complete la section 12. Elle fournit, pour chaque article, un lien direct vers le PDF. Le script de l'annexe C telecharge automatiquement l'ensemble dans *resources/papers/*.

Etat de verification. Les liens marques **Verifie** ont ete controles par recherche. Les autres identifiants arXiv proviennent de la memoire de l'assistant : ouvrir chaque PDF et confirmer que le titre et les auteurs correspondent avant de citer.

#	Article	Lien PDF	Etat
1	U-Net - Ronneberger, Fischer, Brox (2015)	arxiv.org/pdf/1505.04597	A confirmer
2	ResNet - He et al. (2016)	arxiv.org/pdf/1512.03385	A confirmer
3	YOLO - Redmon et al. (2016)	arxiv.org/pdf/1506.02640	A confirmer
4	Faster R-CNN - Ren et al. (2015)	arxiv.org/pdf/1506.01497	A confirmer
5	Mask R-CNN - He et al. (2017)	arxiv.org/pdf/1703.06870	A confirmer
6	Revue des architectures YOLO - Terven, Cordova-Esparza (2023)	arxiv.org/pdf/2304.00501	A confirmer
7	Segment Anything - Kirillov et al. (2023)	arxiv.org/pdf/2304.02643	A confirmer
8	Vision Transformer - Dosovitskiy et al. (2021)	arxiv.org/pdf/2010.11929	A confirmer
9	Swin Transformer - Liu et al. (2021)	arxiv.org/pdf/2103.14030	A confirmer
10	SegFormer - Xie et al. (2021)	arxiv.org/pdf/2105.15203	**Verifie**
11	MLOps : vue d'ensemble - Kreuzberger, Kuhl, Hirschl (2023)	arxiv.org/pdf/2205.02302	A confirmer
12	Jeu de donnees PV - Bradbury et al. (2016), acces libre	pmc.ncbi.nlm.nih.gov/articles/PMC5148580/	**Verifie**
13	Inventaire mondial du PV - Kruitwagen et al. (2021), Nature	www.nature.com/articles/s41586-021-03957-7.pdf	Essai (editeur)
14	Hidden Technical Debt - Sculley et al. (2015), NeurIPS	papers.nips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf	Essai (editeur)

Non disponibles en libre acces via le script : Malof et al. (2016), DeepSolar (Yu et al., 2018), Maggiori et al. (2017), Ji et al. (2019), Zaharia et al. (2018). Les chercher par titre sur Google Scholar ou via l'acces universitaire. Les outils (Ultralytics YOLO11, FastAPI, DVC, MLflow, ONNX Runtime, PVGIS) se citent via leur documentation officielle : https://docs.ultralytics.com/models/yolo11/.

Annexe B - Jeux de donnees et sources

Remplace et precise la section 5.2. Les licences doivent etre relues avant tout usage ou redistribution.

Dataset	Contenu / usage dans le projet	Lien	Etat
Inria Aerial Image Labeling	Batiments (classe unique), 0,3 m/pixel, images 5000x5000, 180 images d'entrainement, villes d'entrainement et de test differentes. Base de la segmentation de toitures. Tres volumineux : decouper en tuiles 512x512.	project.inria.fr/aerialimagelabeling miroir Hugging Face	**Verifie**. Usage recherche, citation d'Inria
Bradbury PV	19 433 panneaux solaires (polygones) sur 601 images de 4 villes de Californie. Utile pour la detection de panneaux existants et l'extension du projet.	figshare.com/collections/3255643	**Verifie**

Dataset	Contenu / usage dans le projet	Lien	Etat
WHU Building	Batiments, imagerie aerienne et satellite, complement pour la generalisation.	gpcv.whu.edu.cn	A verifier (lien, licence)
SpaceNet	Empreintes de batiments sur images satellite, hebergees sur AWS S3.	https://spacenet.ai	A verifier
PVGIS	Irradiation annuelle G (kWh/m2/an) pour Tunis : remplace l'ordre de grandeur de 1800 de la section 5.4.	re.jrc.ec.europa.eu/pvg_tools	A verifier
Jeu local Tunisie	Captures cartographiques annotees par l'equipe, uniquement pour evaluer la generalisation.	Interne	A constituer (licence des fonds de carte)

Limites a connaitre. Inria ne contient qu'une classe (batiment / non-batiment) : les cheminees, climatiseurs et ombres doivent etre pre-annotes avec Segment Anything puis corriges (section 11). Les masques Inria doivent aussi etre convertis en polygones au format YOLO dans l'etape *prepare* de DVC.

Annexe C - Script de telechargement (download_resources.py)

Le script telecharge les articles de l'annexe A et le dataset Bradbury (API figshare), ecrit un *MANIFEST.json* (taille et SHA-256) et affiche les instructions pour les jeux a recuperer manuellement. Il n'utilise que la bibliotheque standard ; *huggingface_hub* est requis seulement pour l'option *--inria*.

Utilisation

```
python download_resources.py --all --dry-run      # simulation
python download_resources.py --papers             # 14 articles PDF
python download_resources.py --datasets           # Bradbury PV
python download_resources.py --datasets --inria   # + Inria (dizaines de Go)
dvc add resources/datasets                        # versionner avec DVC
```

Integration dans la chaine MLOps

Executer le script une fois, puis *dvc add* sur *resources/datasets* : le dossier est ignore par Git et trace par DVC. L'etape *prepare* de *dvc.yaml* (tuilage 512x512 et conversion masques vers polygones YOLO) part de ce dossier.

> Remarque : le script a ete teste en simulation et pour la syntaxe uniquement. Les telechargements reels doivent etre lances depuis votre machine (acces reseau a arXiv, figshare, nature.com). Les echecs sont affiches sans interrompre l'execution.

Code complet

```
#!/usr/bin/env python3
"""SolarVision-MLOps - telechargement des articles scientifiques et des datasets.

Usage :
    python download_resources.py --papers                # 12 articles PDF
    python download_resources.py --datasets              # Bradbury PV (figshare)
    python download_resources.py --datasets --inria      # + Inria (tres volumineux)
    python download_resources.py --all --out resources   # tout (sauf Inria)
    python download_resources.py --all --dry-run         # affiche sans telecharger

Le script n'utilise que la bibliotheque standard (huggingface_hub est optionnel,
seulement pour --inria). Les echecs sont signales sans arreter le script ;
un fichier MANIFEST.json (taille + SHA-256) est ecrit pour DVC / la soutenance.
"""
import argparse
import hashlib
import json
import sys
import time
import urllib.request
from pathlib import Path

UA = "Mozilla/5.0 (SolarVision-MLOps student project)"

# (nom de fichier, URL). Les identifiants arXiv sont a verifier en ouvrant le PDF.
PAPERS = [
    ("01_UNet_Ronneberger_2015", "https://arxiv.org/pdf/1505.04597"),
    ("02_ResNet_He_2016", "https://arxiv.org/pdf/1512.03385"),
    ("03_YOLO_Redmon_2016", "https://arxiv.org/pdf/1506.02640"),
    ("04_FasterRCNN_Ren_2015", "https://arxiv.org/pdf/1506.01497"),
    ("05_MaskRCNN_He_2017", "https://arxiv.org/pdf/1703.06870"),
    ("06_YOLO_review_Terven_2023", "https://arxiv.org/pdf/2304.00501"),
    ("07_SegmentAnything_Kirillov_2023", "https://arxiv.org/pdf/2304.02643"),
    ("08_ViT_Dosovitskiy_2021", "https://arxiv.org/pdf/2010.11929"),
    ("09_SwinTransformer_Liu_2021", "https://arxiv.org/pdf/2103.14030"),
    ("10_SegFormer_Xie_2021", "https://arxiv.org/pdf/2105.15203"),
    ("11_MLOps_Kreuzberger_2023", "https://arxiv.org/pdf/2205.02302"),
]
# Hors arXiv : essai "au mieux" (liens editeurs, peuvent changer ou etre bloques).
OPTIONAL_PAPERS = [
    ("12_Bradbury_PV_dataset_2016",
     "https://www.nature.com/articles/sdata2016106.pdf"),
    ("13_Kruitwagen_global_PV_2021",
     "https://www.nature.com/articles/s41586-021-03957-7.pdf"),
    ("14_Sculley_TechnicalDebt_2015",
     "https://papers.nips.cc/paper_files/paper/2015/file/"
     "86df7dcfd896fcaf2674f757a2463eba-Paper.pdf"),
]

FIGSHARE_COLLECTION = 3255643  # Bradbury et al., panneaux PV (Californie)
INRIA_HF_REPO = "usmanhf/INRIA-Aerial-Image-Labeling"
MANUAL = """
```

```
Datasets a telecharger manuellement (licences / formulaires) :
  - Inria Aerial (officiel) : https://project.inria.fr/aerialimagelabeling/
  - WHU Building            : http://gpcv.whu.edu.cn/data/building_dataset.html
  - SpaceNet (AWS, sans compte) :
       aws s3 ls s3://spacenet-dataset/ --no-sign-request
       aws s3 cp s3://spacenet-dataset/<chemin> data/raw/spacenet/ --recursive --no-sign-request
  - Irradiation G (PVGIS)   : https://re.jrc.ec.europa.eu/pvg_tools/en/
Verifier la licence de chaque jeu avant utilisation.
"""


def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def download(url: str, dest: Path, pdf: bool = False, retries: int = 3) -> str:
    if dest.exists() and dest.stat().st_size > 1000:
        return "deja present"
    last = None
    for attempt in range(retries):
        try:
            req = urllib.request.Request(url, headers={"User-Agent": UA})
            tmp = dest.with_suffix(dest.suffix + ".part")
            with urllib.request.urlopen(req, timeout=60) as r, tmp.open("wb") as f:
                first = r.read(1 << 20)
                if pdf and not first.startswith(b"%PDF"):
                    raise ValueError("la reponse n'est pas un PDF")
                f.write(first)
                while True:
                    chunk = r.read(1 << 20)
                    if not chunk:
                        break
                    f.write(chunk)
            tmp.replace(dest)
            return "ok"
        except Exception as exc:  # noqa: BLE001 - on veut continuer quoi qu'il arrive
            last = exc
            time.sleep(2 * (attempt + 1))
    return f"ECHEC ({last})"


def get_json(url: str):
    req = urllib.request.Request(url, headers={"User-Agent": UA})
    with urllib.request.urlopen(req, timeout=60) as r:
        return json.load(r)


def do_papers(out: Path, dry: bool, manifest: dict):
    folder = out / "papers"
    folder.mkdir(parents=True, exist_ok=True)
    for name, url in PAPERS + OPTIONAL_PAPERS:
        dest = folder / f"{name}.pdf"
        if dry:
            print(f"[dry-run] {url} -> {dest}")
            continue
        status = download(url, dest, pdf=True)
        print(f"{status:<40} {name}")
        if dest.exists():
            manifest[str(dest)] = {"bytes": dest.stat().st_size,
                                   "sha256": sha256(dest), "url": url}


def do_bradbury(out: Path, dry: bool, max_mb: int, manifest: dict):
    folder = out / "datasets" / "bradbury_pv"
    folder.mkdir(parents=True, exist_ok=True)
    api = "https://api.figshare.com/v2"
    try:
        articles = get_json(f"{api}/collections/{FIGSHARE_COLLECTION}/articles"
                            "?page_size=100")
    except Exception as exc:  # noqa: BLE001
        print(f"ECHEC liste figshare : {exc}")
        return
    for art in articles:
        try:
            info = get_json(f"{api}/articles/{art['id']}")
        except Exception as exc:  # noqa: BLE001
            print(f"ECHEC article {art['id']} : {exc}")
            continue
        for fi in info.get("files", []):
            size_mb = fi.get("size", 0) / 1e6
            dest = folder / fi["name"]
            if size_mb > max_mb:
                print(f"IGNORE (> {max_mb} Mo) {fi['name']} ({size_mb:.0f} Mo)")
                continue
            if dry:
                print(f"[dry-run] {fi['download_url']} -> {dest}")
                continue
```

```
            status = download(fi["download_url"], dest)
            print(f"{status:<40} {fi['name']} ({size_mb:.1f} Mo)")
            if dest.exists():
                manifest[str(dest)] = {"bytes": dest.stat().st_size,
                                       "sha256": sha256(dest),
                                       "url": fi["download_url"]}


def do_inria(out: Path, dry: bool):
    folder = out / "datasets" / "inria"
    print("ATTENTION : Inria est tres volumineux (dizaines de Go). "
          "Prevoyez de l'espace disque.")
    if dry:
        print(f"[dry-run] huggingface_hub.snapshot_download({INRIA_HF_REPO}) -> {folder}")
        return
    try:
        from huggingface_hub import snapshot_download
    except ImportError:
        print("Installez d'abord : pip install huggingface_hub")
        return
    snapshot_download(repo_id=INRIA_HF_REPO, repo_type="dataset",
                      local_dir=str(folder))
    print(f"Inria telecharge dans {folder}")


def main() -> int:
    p = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--out", default="resources", help="dossier de sortie")
    p.add_argument("--papers", action="store_true", help="telecharger les articles")
    p.add_argument("--datasets", action="store_true", help="telecharger Bradbury PV")
    p.add_argument("--inria", action="store_true", help="telecharger Inria (tres gros)")
    p.add_argument("--all", action="store_true", help="articles + datasets")
    p.add_argument("--max-mb", type=int, default=500,
                   help="ignorer les fichiers figshare plus gros (defaut 500)")
    p.add_argument("--dry-run", action="store_true", help="afficher sans telecharger")
    args = p.parse_args()

    if not (args.papers or args.datasets or args.inria or args.all):
        p.print_help()
        return 1

    out = Path(args.out)
    manifest: dict = {}
    if args.papers or args.all:
        do_papers(out, args.dry_run, manifest)
    if args.datasets or args.all:
        do_bradbury(out, args.dry_run, args.max_mb, manifest)
    if args.inria:
        do_inria(out, args.dry_run)
    print(MANUAL)

    if manifest and not args.dry_run:
        mpath = out / "MANIFEST.json"
        mpath.write_text(json.dumps(manifest, indent=2, ensure_ascii=False))
        print(f"Manifeste ecrit : {mpath}")
        print("Versionner avec DVC :  dvc add resources/datasets")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Annexe D - Mises a jour par rapport a la premiere version

Points du rapport initial qui sont precises ou corriges par ces annexes :

- **Avertissement (page 1) et section 12 :** deux references (SegFormer, Bradbury) et deux jeux de donnees (Inria, Bradbury) sont desormais verifies par recherche ; les autres restent a confirmer, comme indique en annexe A.
- **Section 5.2 (donnees) :** remplacee par l'annexe B, avec liens et limites (classe unique d'Inria, volume).
- **Section 5.4 (energie) :** G = 1800 kWh/m2/an reste un ordre de grandeur ; a remplacer par la valeur PVGIS pour Tunis.
- **Section 7 (chaine MLOps) :** l'etape de collecte des donnees est desormais automatisee par le script de l'annexe C et tracee par DVC.
- **Calendrier et bareme :** inchanges, a recaler sur le cahier des charges officiel du module.